

哈 尔 滨 工 业 大 学

## 硕士学位论文开题报告

题 目：移位异或纠删码的高效实现研究

|        |                  |            |          |
|--------|------------------|------------|----------|
| 院      | (系)              | 计算机科学与技术学院 |          |
| 学      | 科/专              | 业          | 计算机科学与技术 |
| 导      | 师                | 王强副教授      |          |
| 研      | 究                | 生          | 孙铎       |
| 学      | 号                | 24S051046  |          |
| 开题报告日期 | 2025 年 08 月 30 日 |            |          |

深圳校区教务部 制

# 目 录

|                                                  |    |
|--------------------------------------------------|----|
| 第 1 章 课题的来源及研究目的和意义 .....                        | 1  |
| 1.1 课题来源 .....                                   | 1  |
| 1.2 研究的目的和意义 .....                               | 2  |
| 第 2 章 国内外研究现状及分析 .....                           | 3  |
| 2.1 国内外研究现状 .....                                | 3  |
| 2.2 国内外文献综述及简析 .....                             | 3  |
| 2.2.1 RS 码相关研究分析 .....                           | 3  |
| 2.2.2 移位异或码相关研究分析 .....                          | 4  |
| 第 3 章 主要研究内容及研究方案 .....                          | 6  |
| 3.1 研究内容 .....                                   | 6  |
| 3.1.1 存储编码的基本原理 .....                            | 6  |
| 3.1.2 Two-tone Shift-XOR 的编解码原理 .....            | 7  |
| 3.1.3 优化问题描述 .....                               | 11 |
| 3.2 研究方案 .....                                   | 11 |
| 3.2.1 统一解码抽象: Erasure-Parity Mapping .....       | 11 |
| 3.2.2 缓存友好的窗口划分: Cache-Friendly Window .....     | 13 |
| 3.2.3 内存友好的相邻合并访问: Neighbor-Merging Access ..... | 14 |
| 3.2.4 特化与泛化结合: Special Case Expansibility .....  | 15 |
| 第 4 章 预期目标 .....                                 | 17 |
| 4.1 系统集成与应用目标 .....                              | 17 |
| 4.1.1 接口集成与功能验证 .....                            | 17 |
| 4.1.2 集群集成与应用验证 .....                            | 17 |
| 4.2 性能优化与方法论总结目标 .....                           | 17 |
| 第 5 章 已完成的研究工作及进度安排 .....                        | 19 |
| 5.1 已完成的研究工作 .....                               | 19 |
| 5.2 进度安排 .....                                   | 19 |
| 第 6 章 已具备的研究条件和所需条件及经费 .....                     | 20 |
| 6.1 实验室条件和经费保障 .....                             | 20 |
| 6.2 所需条件及经费 .....                                | 20 |
| 第 7 章 预计困难及解决方案 .....                            | 21 |
| 7.1 预计困难与技术难点 .....                              | 21 |
| 7.2 解决方案 .....                                   | 21 |
| 参考文献 .....                                       | 23 |

# 第 1 章 课题的来源及研究目的和意义

## 1.1 课题来源

在现代存储系统中，数据可用性对于应对各种故障至关重要<sup>[1]</sup>。目前，主流的容错技术包括数据复制和纠删码。数据复制通过在不同节点保存多份副本，虽然其实现简单，但是存储开销较大。例如，三副本机制能同时容忍小于等于两个故障，但是需要三倍的存储空间。相比之下，纠删码技术通过对数据进行分块编码和解码，能够以远低于复制的存储成本，提供同等甚至更高的数据可靠性<sup>[2]</sup>。现如今，纠删码已成为 Google 文件系统（GFS）<sup>[3]</sup>、Windows Azure 存储<sup>[4,5]</sup>、Hadoop 分布式文件系统（HDFS）<sup>[6,7]</sup> 以及 Ceph<sup>[8,9]</sup> 等大规模存储系统的重要技术。

纠删码通过对原始数据分块进行编码生成冗余校验信息，从而提升数据可靠性，并在数据丢失时实现恢复。与复制技术相比，经典纠删码 Reed-Solomon 码（RS 码）<sup>[10]</sup> 的编码和解码过程计算量更大<sup>[2]</sup>，其通常依赖于有限域  $GF[2^w]$ ,  $w > 1$  上的矩阵乘法，并在计算时将部分有限域上的运算转换为异或运算（亦即有限域  $GF[2]$  上的运算）<sup>[11]</sup> 从而提高计算速度。众多研究如 Jerasure<sup>[12,13]</sup>、G-CRS<sup>[14]</sup>、Zerasure<sup>[15,16]</sup>、SLPEC<sup>[17]</sup>、Cerasure<sup>[18,19]</sup> 和 Intel 的 ISA-L 库<sup>[20]</sup> 等都致力于提升基于有限域矩阵乘法的编解码效率，以降低系统成本并增强服务可靠性。近年来，基于非循环移位异或运算的纠删码在理论层面取得许多进展，例如 ZigZag Decodable Codes<sup>[21-23]</sup> 和 Two-tone Shift-XOR Codes<sup>[24-28]</sup>，并且在存储<sup>[23,29]</sup>、密码学<sup>[30-32]</sup> 和网络<sup>[33]</sup> 等应用领域进行了各种拓展研究。对于这类移位异或的编解码算法来说，异或运算就是最基本的运算，而并非像传统纠删码那样需要转化，这种更简单更直接的运算方式也取得了更高的理论性能<sup>[23]</sup>。相比于 ZigZag 算法，Two-tone 算法对于移位和异或操作的编排更具有规则性，使得其在解码时无需多次扫描即可解出数据的位置，更容易按顺序访问内存，在算法实现层面有更高效率的潜能<sup>[27,28]</sup>。

尽管 Two-tone 移位异或纠删码算法在理论上具备优越性能，其完整的工程实现与系统级优化方面的工作尚且缺少。现有文献虽展示了算法潜力，但缺乏可用于生产环境的提速方案和高性能实现。具体来讲，可总结为以下三点问题：

- 缺少泛化性实现：当前的算法实现只能在实验测试中证明高效性，但并不具有泛化能力，无法应对数据块和校验块（存储冗余信息的块）所有的丢失情况，不能同时满足存储系统中解码丢失数据块和重新编码丢失校验块的双重需求。
- 缺少系统级优化：理论算法没有明确给出程序优化的策略，例如减少条件

分支和遍历次数、增强缓存局部性和减轻内存访问压力等方面的优化，同时 RS 纠删码的系统级优化策略因其基本算法不同而无法直接应用到移位异或纠删码上。

- 缺少工业级应用：上述纠删码研究成果仅有 Jerasure 和 ISA-L 等少数基于 RS 码的算法以插件的形式集成到了 Ceph 和 Hadoop 等开源存储系统中，大部分纠删码（包括移位异或纠删码）目前并没有在工业级应用上测试并验证高效性。

鉴于上述三点问题，本课题将 Two-tone 非循环移位异或纠删码以插件形式集成至开源分布式存储系统 Ceph 中，作为其存储池管理的一种存储编码，借助 Ceph 中的应用更加规范地对比测试纠删码算法，并在该环境下应用各种系统级优化方案，提炼出针对移位异或纠删码的高效实现方法论，验证各方法论的有效性，达成比 Ceph 中的 SOTA 纠删码插件和最新研究中的 SOTA 纠删码算法更高效的性能。

## 1.2 研究的目的和意义

本课题旨在设计高效的 Two-tone 非循环移位异或纠删码实现方案，推动这类新型移位异或纠删码算法在大规模分布式存储系统例如 Ceph 中的落地应用，实现比其他纠删码算法更加优秀的应用性能。通过理论算法与工程实践的创新性结合，力求进一步提高纠删码在实际应用中的性能，让大规模分布式存储系统更加高效。

本课题的研究目的一是针对 Two-tone 移位异或纠删码算法本身在程序实现时的瓶颈，提出系统级优化方法，包括减少条件分支和循环遍历次数、提升缓存命中率、优化内存访问模式等，提炼并总结适用于新型移位异或纠删码的优化方案，为后续更复杂深入的优化打下基础；二是将 Two-tone 纠删码以插件的形式集成至 Ceph 分布式存储系统中，通过完整的功能性测试，验证各优化方案的有效性，并实施性能对比实验，与 Ceph 中的 SOTA 纠删码 ISA-L 比较，与 RS 码的 SOTA 实现 Cerasure 和移位异或码的 SOTA 实现 ZigZag 比较，取得更高效的性能。

本课题的研究意义一是推动非循环移位异或纠删码从算法理论到实际应用的发展，呈现新型高效纠删码在存储系统中的解决方案，丰富存储系统的编码手段，并实现更高效的纠删码算法，进一步提高编解码吞吐量，促进解决纠删码吞吐量不匹配高性能设备的问题<sup>[34]</sup>，为分布式存储系统的性能提升提供技术支撑；二是填补移位异或纠删码缺少系统级优化方法论的空白，对齐 RS 码的研究进度，发挥出移位异或码的高效潜能，同时为开源存储系统 Ceph 提供此前未有的高性能移位异或纠删码插件，促进开源社区的发展，推动相关领域的技术进步。

## 第 2 章 国内外研究现状及分析

### 2.1 国内外研究现状

随着分布式存储系统的快速发展，纠删码作为容错和数据可靠性保障的核心技术，受到了广泛关注。传统的 Reed-Solomon (RS) 码<sup>[10]</sup> 在理论创新和系统优化层面都取得了重要进展，其基于有限域的编码方式具有良好的 MDS（最大距离可分）特性，能够保证在容忍范围内丢失时恢复原始数据。在此基础上，将有限域运算转化为异或运算（XOR-Based）以及采用 Vandermonde 或 Cauchy 矩阵以减少逆运算开销进一步提升了 RS 码的理论编解码效率<sup>[11]</sup>。在系统实现上，优化矩阵结构以减少异或操作数和提取并复用公共运算表达式等技术不断涌现，综合各优化方案的 Cerasure<sup>[19]</sup> 显著提高了编码吞吐量，在一定条件下取得了 SOTA 成果。

近年来，基于非循环移位异或运算的新型纠删码逐渐成为新的理论研究热点。ZigZag Decodable Codes<sup>[23]</sup> 通过构建特殊的移位矩阵，实现了仅依赖异或和移位操作的高效编码与解码，理论上可达到线性时间复杂度，并在存储、密码学和网络等领域得到拓展。Two-tone Shift-XOR Codes<sup>[28]</sup> 进一步提升了移位异或码的结构规则性和解码效率，具备更高的吞吐性能。在测试环境的特定条件下，Two-tone 编码可实现比 RS 码更高的编码和解码速度，尤其适用于高性能分布式存储系统。

### 2.2 国内外文献综述及简析

#### 2.2.1 RS 码相关研究分析

RS 码是最经典的纠删码，最早由 Reed 和 Solomon 提出<sup>[10]</sup>，其理论基础和 MDS 特性为后续编码技术奠定了坚实基础。Luby 等人提出了基于异或运算的优化方法<sup>[11]</sup>，并使用求逆运算更快的 Cauchy 矩阵代替 Vandermonde 矩阵，达到 RS 码理论最优性能，后续研究工作都依赖于这项成果。随着硬件并行能力提升，Jerasure<sup>[12,13]</sup> 和 G-CRS<sup>[14]</sup> 等实现了 CPU 多线程并发和 GPU 加速，但对硬件资源有特定要求。

为进一步提高 RS 码吞吐量，研究者们陆续提出了许多系统级优化实现策略：

- 减少比特矩阵中 1 的个数进而减少运算：[35] 开发 “Good Cauchy (GC)” 矩阵优化生成序列，从而最小化 1 的个数；[36] 通过整数线性规划和分支定界等算法优化 Cauchy 矩阵的生成序列组合从而降低 1 的密度；Zerasure<sup>[16]</sup> 通过智能算法优化 Cauchy 生成序列，最小化权重即 1 的数量；Cerasure<sup>[19]</sup> 提出 V-search 算法同时

搜索 Cauchy 和 Vandermonde 矩阵的特殊构造，得到含最少非零元素的编码矩阵。

- 提取并复用公共异或运算表达式：[37] 引入 X-Sets 数据结构动态识别公共子表达式，再结合 Uber-CSHR 参数化扩展复用异或运算；Zerasure<sup>[16]</sup> 应用匹配算法构建需求图，识别高频数据位对，通过贪婪策略提取公共 XOR 链；SLPEC<sup>[17]</sup> 将编码过程转化为直线程序（SLP），利用扩展的 XorRePair 算法识别并复用重复的异或表达式，再结合 XOR 的幂等性消除冗余计算；Cerasure<sup>[19]</sup> 采用智能调度技术和拆解编码计算，从而尽可能多地复用计算的公共中间结果。

- 内存与缓存访问优化：[38] 通过引入数据导向的 XOR 调度算法重新排序 XOR 操作的执行顺序以增强数据局部性；Zerasure<sup>[16]</sup> 采用顺序数据访问策略及相关调度器提升空间局部性；SLPEC<sup>[17]</sup> 采用 Deforestation 技术 消除中间数组分配，通过分块策略 划分数据块，最小化缓存冲突并减少内存访问；Cerasure<sup>[19]</sup> 的智能调度和编码拆解还可用于让局部的计算适配缓存大小从而提升缓存命中率。

- 降低数据指针相关开销：Cerasure<sup>[19]</sup> 使用 二维数组存储指针结合 SIMD 指令提出地址计算模式，从而减少反复构造和操作指针引发的额外开销。

正如上文所述，在这些系统实现优化研究中，Cerasure 综合了所有常见策略，给出了更进一步的优化方式，并提出了降低数据指针相关开销的优化方案，是目前基于有限域矩阵乘法的纠删码（RS 码）系统级优化的 SOTA 实现。

### 2.2.2 移位异或码相关研究分析

ZigZag Decodable Codes (ZD Codes) 是较早提出的移位异或码，首先验证了基于 Vandermonde 矩阵构建移位矩阵并异或计算校验块的可行性与高效性<sup>[21]</sup>，又在此基础上换用 Hankel 矩阵构建更高效的算式，进一步提高编解码效率<sup>[23]</sup>。在存储应用拓展方面，[22] 提出新型循环矩阵结构，进一步降低存储开销，并通过物理层编码实现无线分布式存储的高效修复，显著降低节点修复带宽；[29] 设计最优带宽重建方案，其子包抓取算法使带宽消耗严格等于待重建数据量，消除额外开销，适用于高密度存储系统。在网络应用拓展方面，[33] 将 ZD Codes 应用于无线多址接入场景，允许多个发射机同时传输数据并使用 ZigZag Decoding 解析叠加信号，突破传统系统吞吐量的限制。在密码学应用拓展方面，[30] 利用 ZD Codes 构建门限秘密共享方案，通过 Vandermonde 矩阵生成编码多项式并截断处理，实现完美保密性和高效解码；[31] 提出新型生成矩阵降低 ZD Codes 存储开销，通过重构半 Vandermonde 半循环矩阵结构减少存储开销并证明其解码性和安全性，扩展了其在资源受限场景的应用边界；[32] 将 ZD Codes 应用于分布式存储的部分数据

保护，提出轻量级秘密共享方案，实现低存储开销和小修复度，同时支持并行解码加速，显著提升分布式存储的可靠性和修复效率。

Two-tone Shift-XOR Codes 的提出是基于对移位异或纠删码理论本质的探索，主要体现在算法规范化上。[24] 和 [25] 初步提出移位异或纠删码的递增差 (Increasing-Difference) 性质，保证编码移位的偏移量的可解性与解码逻辑的正确性，并探索了理想情况下原地解码的可行性。[26] 进一步明确改进的递增差 (Refined Increasing-Difference) 性质，优化偏移量递增差的约束，允许更灵活的设计，降低存储额外开销至常数级，还提出了移位异或消除法 (Shift-XOR Elimination)，可以直接在接收缓冲区上进行解码运算，逐层消除并解出所有数据块。[27] 提出连续可解性 (Successive Solvability) 理论，更加规范地定义移位矩阵，从而保证解码时可以按索引顺序依次解码，不再需要每次都扫描全部索引，简化了移位异或纠删码的解码，也提供了移位异或系统可解性的统一且可判定的理论框架。

在上述移位异或理论的基础上，Two-tone 纠删码算法<sup>[28]</sup> 得以正式提出。Two-tone 突破了一般 RID 矩阵单调偏移的约束，继承了连续可解性，设计先递减后递增的对称偏移模式，并巧妙地将解码系统划分成两个子系统进行解码，两个子系统分别对应编码时递减和递增的部分，可以规则且高效地完成数据块的解码，达到更高效的理论性能，并且其规范性也使得后续进行系统优化有更大的潜能。

虽然关于 ZD Codes 和 Two-tone Codes 这类移位异或纠删码的理论研究已经相当充分，但关于对其进行系统级优化的工作依然缺少，其落地到工业级应用场景下的优化与实验依然有较大空白。并且 RS 纠删码的系统优化策略无法直接应用到 Two-tone 移位异或纠删码上，因为在移位异或纠删码中所有数据块码字都会参与异或，无法直接使用 RS 码减少异或运算的方法。这是由移位矩阵决定的，也是避免了有限域上其他运算的结果。而编码不同校验块时不同的移位偏移量也导致无公共表达式，偏移量计算方式的不同以及数据访问模式的不同也无法直接应用 RS 码内存缓存和指针开销的优化策略。因此，针对移位异或码的系统级优化方法仍需进一步探索和完善。本课题正是基于更规范高效的 Two-tone 移位异或纠删码，研究其系统优化策略，填补移位异或纠删码在系统优化方法论上的空白。

## 第 3 章 主要研究内容及研究方案

### 3.1 研究内容

#### 3.1.1 存储编码的基本原理

纠错码是一类通过冗余编码提升数据可靠性的算法，可作为存储编码应用到存储系统中，相比于创建多个数据副本防丢失，它通过数学算法将原始数据转换为带冗余信息的编码数据，在可靠性与存储效率间取得平衡，适用于大规模分布式存储场景。以最经典的 Reed-Solomon (RS) 码<sup>[10]</sup>为例，假设将原始数据均分成  $k$  个数据块，再同生成矩阵进行有限域上的矩阵乘法运算，编码出  $m$  个校验块，总计得到  $n = k + m$  个块，并将这  $n$  个块分别存储到  $n$  台分布式的存储设备中，这样最多能容忍  $m$  块数据丢失，即  $m$  台设备故障。RS 码的编码过程可由式 (3-1) 表示。

$$\mathbf{C} = \mathbf{G} \times_{GF[2^w]} \mathbf{D} \quad (3-1)$$

其中,  $\mathbf{D}$  为原始数据块向量,  $\mathbf{G}$  为生成矩阵(通常为 Vandermonde 矩阵或 Cauchy 矩阵<sup>[11]</sup>),  $\mathbf{C}$  为编码后的校验块向量。图3-1以  $k = 4, m = 3$  为例, 展示了 RS 码的编码过程, 图中的  $D_i$ 、 $C_i$  和  $g_{ij}$  均对应式 (3-1) 中变量的分量。

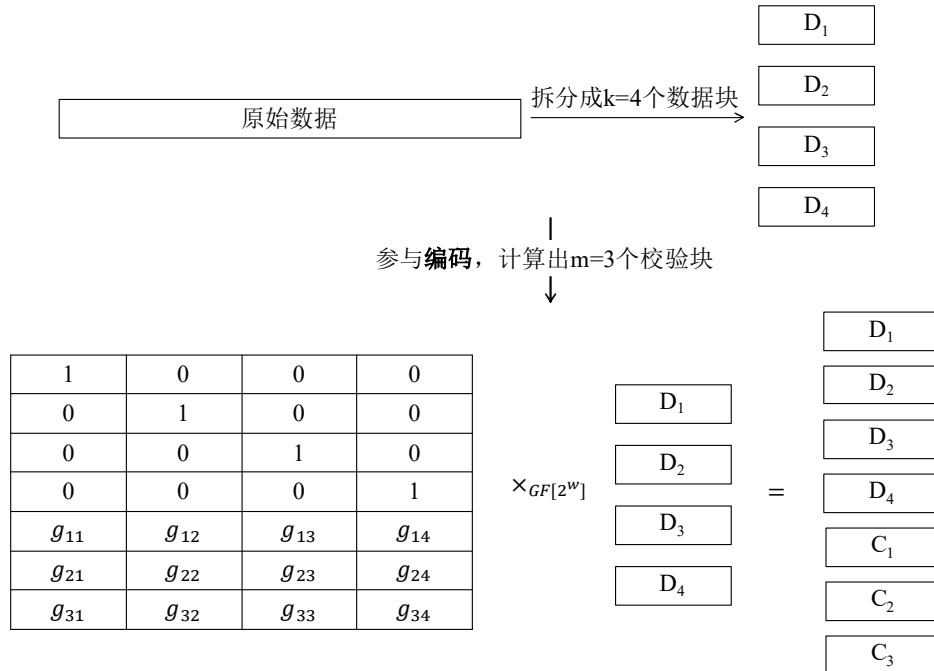


图 3-1 RS 码编码举例



解码时，根据丢失数据块的情况，基于编码公式构造逆矩阵，得出新的解码公式，从而恢复丢失的数据块，再重新编码以恢复丢失的校验块，可由式 (3-2) 表示。

$$\mathbf{D} = \mathbf{G}'^{-1} \times_{GF[2^w]} \mathbf{D}' \quad (3-2)$$

其中， $\mathbf{D}'$  为所有未丢失的数据块和未丢失的校验块构成的向量， $\mathbf{G}'^{-1}$  为根据丢失情况构造的逆生成矩阵， $\mathbf{D}$  为编码前的原始数据块向量。图3-2展示了 RS 码的解码和重编码过程，以丢失两个数据块  $D_2$ 、 $D_4$  和一个校验块  $C_1$  为例。

假设  $D_2$ 、 $D_4$  和  $C_1$  这3个块丢失，在编码公式的结果中去除这三块，得新的公式

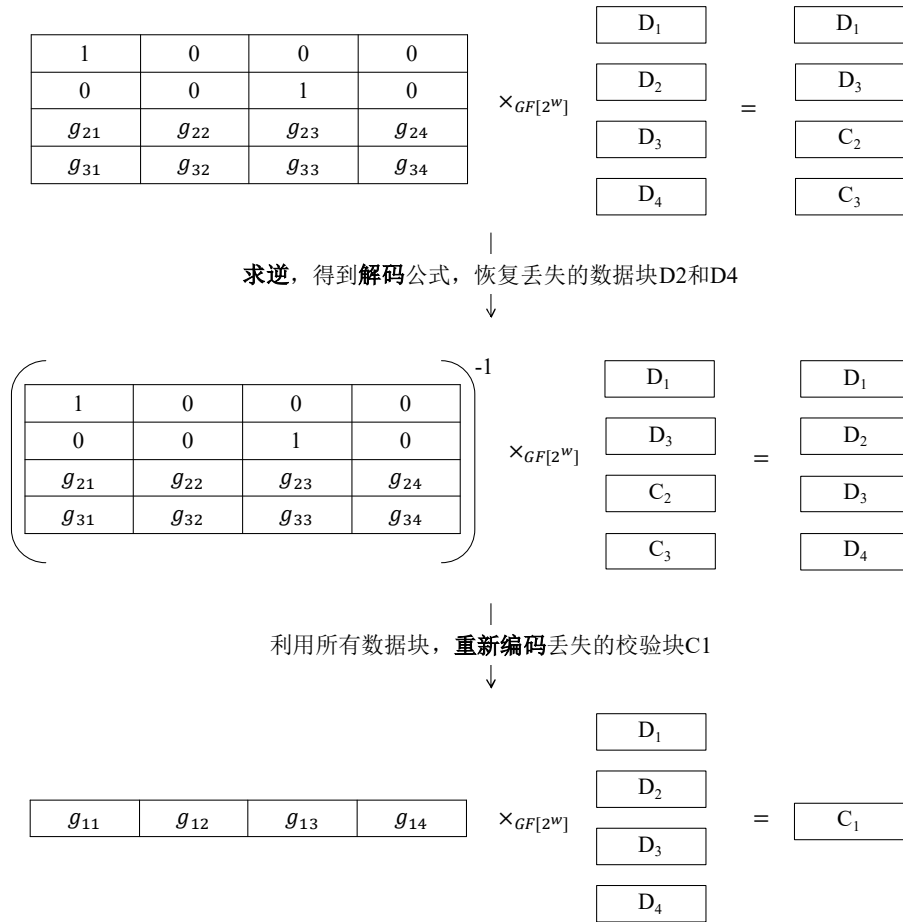


图 3-2 RS 码解码举例

### 3.1.2 Two-tone Shift-XOR 的编解码原理

Two-tone Shift-XOR 码是一种基于非循环移位和异或操作的高效纠删码。其核心思想是利用规则的移位矩阵和异或运算实现编码与解码，避免有限域上的乘法

运算，提升运算效率。Two-tone 码的编码过程可由式 (3-3) 表示。

$$\mathbf{C} = \mathbf{Z} \times \mathbf{D} \quad (3-3)$$

其中， $\mathbf{Z}$  为移位生成矩阵，采用最简单且规则的 Two-tone 矩阵<sup>[28]</sup>，其中每个元素  $z^{s_{ij}}$  代表偏移量。假设原始数据块分别为  $\{D_1, D_2, \dots, D_k\}$ ，编码后得到的校验块分别为  $\{C_1, C_2, \dots, C_m\}$ ，则 Two-tone 的编码公式还可以表示为式 (3-4)。

$$C_i = \bigoplus_{j=1}^k z^{s_{ij}} D_j \quad (3-4)$$

其中  $z^{s_{ij}} D_j$  表示对数据块  $D_j$  进行  $s_{i,j}$  个码字的非循环移位， $\oplus$  为异或操作。图3-3同样以  $k = 4, m = 3$  为例展示了 Two-tone 码的编码过程和移位异或拆解示意。

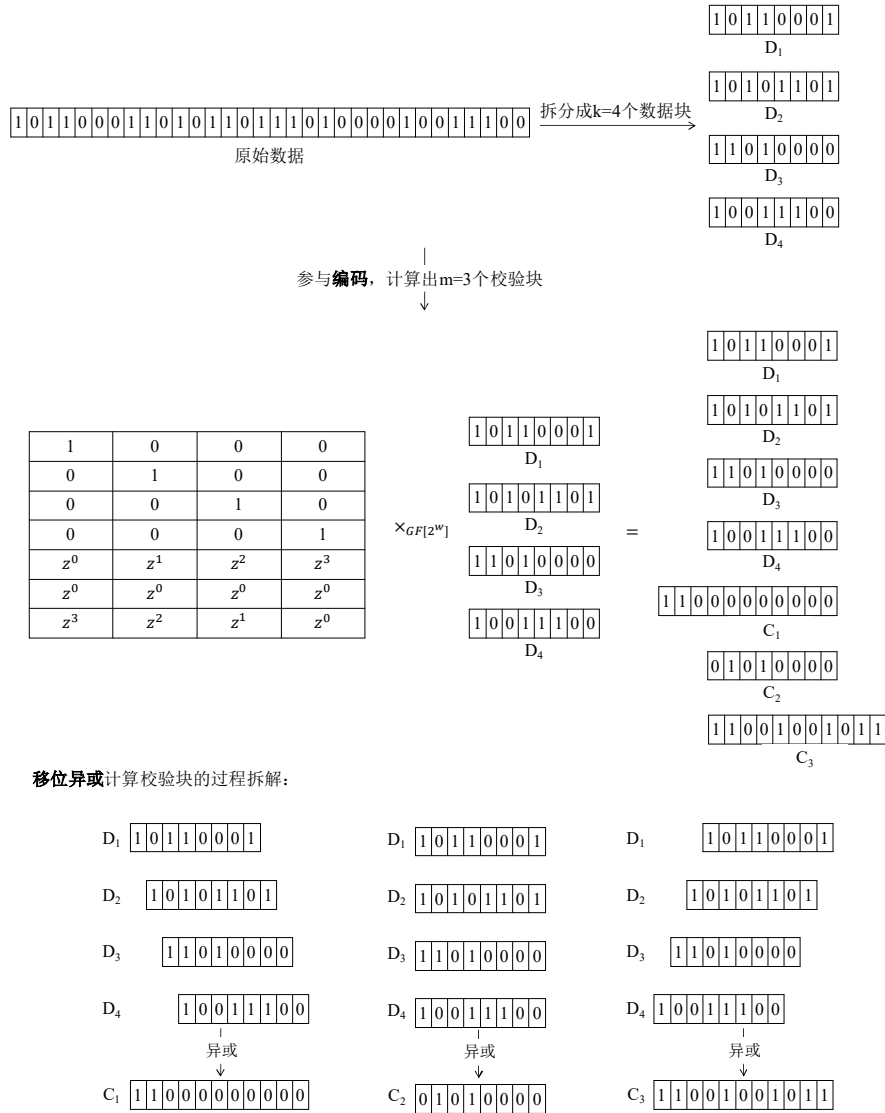


图 3-3 Two-tone 码编码举例

Two-tone 算法的移位生成矩阵具有规则结构，使得解码时可以按序逐步解出丢失块，提升解码效率。图3-4举例展示了 Two-tone 码在丢失三个数据块时的解码过程，总体分成两步，第一步是“代入”，将未丢失的数据块代入至校验块中，第二步是“Two-tone 消除”，每一轮中，从一边最外侧错开的码字开始，依次向内解出每个码字，直到无法再求解，再从另一边最外侧错开的码字开始，依次向内求解，直到解出该轮的所有码字，再进入下一轮，循环解出所有数据。解出的码字也代入校验块，以便后续继续解码。这里提到的“一边”和“另一边”，即对应 Two-tone 算法中的两个子系统<sup>[28]</sup>，按序求解这两个子系统，即可高效解出所有数据。

假设 $D_2$ 、 $D_3$ 和 $D_4$ 这三个块丢失，在编码拆解图中抹除这三块

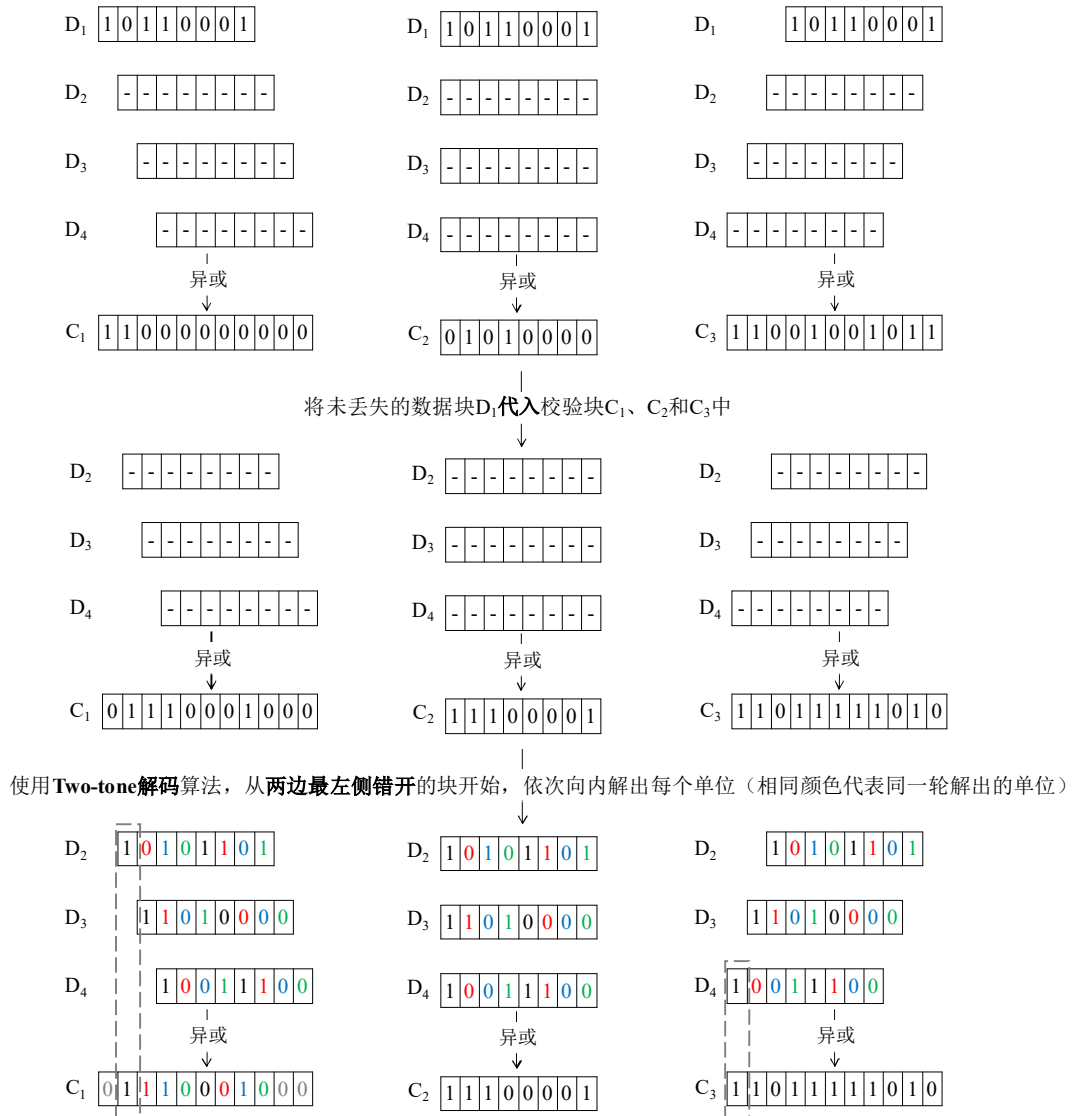
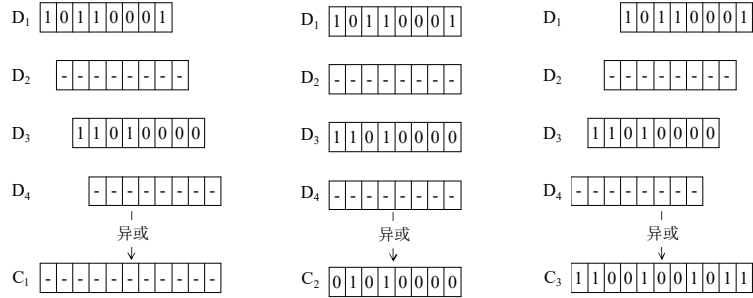


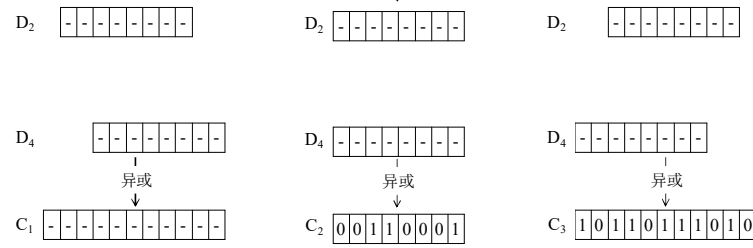
图 3-4 Two-tone 码丢失 3 个数据块的解码举例

图3-5举例展示了 Two-tone 码在丢失两个数据块和一个校验块时的解码过程，不同之处在于，这种情况下只有一边有错开的码字，相当于两个子系统的其中一个为空集，无需进行操作，其他流程不变即可。本例中由于出现了校验块的丢失，在解码完成后，还需要重新编码丢失的校验块，这样才算达成了所有要求。

假设 $D_2$ 、 $D_3$ 和 $C_1$ 这三块丢失，在编码拆解图中抹除这三块



将未丢失的数据块 $D_1$ 和 $D_3$ 代入未丢失的校验块 $C_2$ 和 $C_3$ 中



使用Two-tone解码算法，从最外侧错开的块开始，依次向内解出每个单位（与丢失三块情况类似，只是少了一边）

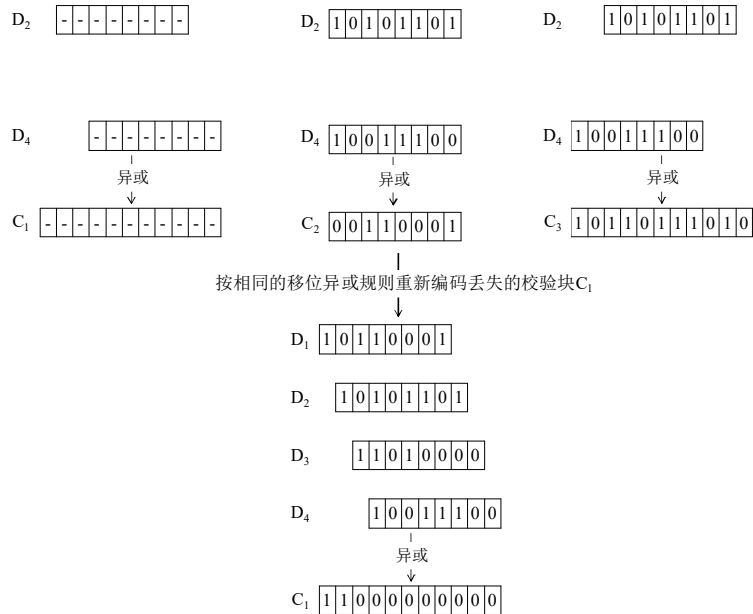


图 3-5 Two-tone 码丢失 2 个数据块和 1 个校验块的解码举例

### 3.1.3 优化问题描述

尽管 Two-tone Shift-XOR 纠删码在理论算法层面具备简洁性和高效性，但在实际的工程实现层面仍面临着如下的系统优化问题：

- 如何统一处理所有丢失情况，实现解码和重编码的泛化，兼顾高效性；
- 如何提升缓存局部性，减少内存访问压力，缓解算法整体的内存瓶颈；
- 如何针对只丢失一块的常见场景进一步特化优化，发挥硬件指令优势。

基于此，本课题给出了一系列移位异或纠删码在系统优化层面的研究方案。

## 3.2 研究方案

### 3.2.1 统一解码抽象：Erasure-Parity Mapping

对于不使用特殊抽象方法的解码，在“代入”步骤时，需要判断校验块是否存在，才能确定是否要执行代入计算。在“消除”步骤时，每个丢失的数据块要按顺序匹配未丢失的校验块，从而按照前文所述的消除规则，从错开的码字开始解码，将解出的码字继续“代入”，进而解出完整的数据块。最后，还需要判断是否要重新编码丢失的校验块。图3-6展示了这种解码方式的总体流程。

针对多样化的丢失情况，本课题提出 Erasure-Parity Mapping 的解决方案。该方案将所有丢失的数据块和丢失的校验块都统一抽象为 Erasure 结构，每个 Erasure 结构按块编号顺序匹配未丢失的校验块，若 Erasure 结构本身就来源于丢失的校验块，则其匹配自己即可。在“代入”步骤时，无需额外判断，直接将未丢失的数据块代入 Erasure 结构即可。在“消除”步骤时，直接从已经匹配好的校验块中解出数据码字即可，将解出的码字同样直接“代入”到所有 Erasure 结构中。因为以上两个步骤是对所有 Erasure 结构的统一操作，而 Erasure 结构包括了所有丢失的数据块和校验块，所以数据块的解码和校验块的重编码可以互相穿插并同时完成，无需单独的重编码步骤，进一步减少了遍历与内存访问次数。

图3-7展示了 Erasure-Parity Mapping 解码方案的总体流程，算法3-1列出了该方案的伪代码实现，其中  $D$  为数据块指针数组， $C$  为校验块指针数组， $Ed$  为丢失的数据块索引， $Ec$  为丢失的校验块索引， $k$  为数据块个数， $m$  为校验块个数，数据块和校验块依次从 0 到  $k + m - 1$  编码。这种 Erasure 结构实现了解码流程的统一抽象，将同类操作统一进行，同时处理码字的解码与重编码，实现了简洁与高效。

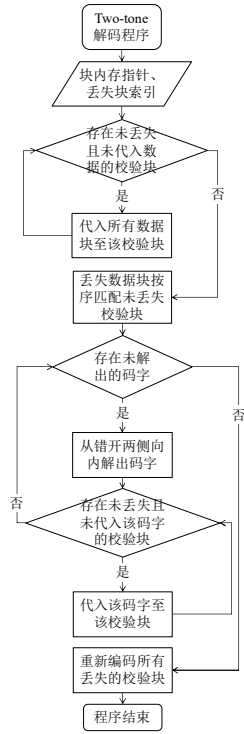


图 3-6 Two-tone 一般解码实现流程

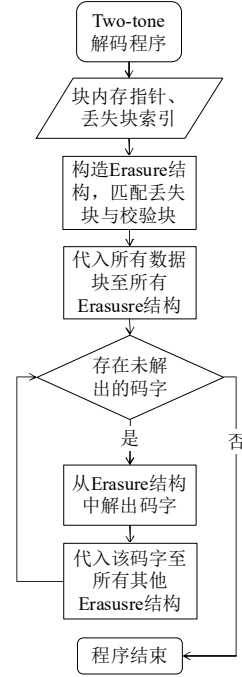


图 3-7 Erasure-Parity Mapping 统一解码实现流程

## 算法 3-1 Two-tone Erasure-Parity Mapping 解码

**Input:**  $D, C$ , Ascendingly sorted  $Ed$  and  $Ec, k, m$   
**Output:** Recover all erased data and coding chunks

```

1  /* Step 1: Map erasures with parities */
2  for  $i \in Ec$  do
3       $map\_erasure\_ptr[i - k] \leftarrow C[i]$  init by 0
4  end
5  for  $i \in (\{k, \dots, k + m - 1\} - Ec)$  and  $!Ed.empty()$  do
6       $coding\_data\_map[i - k] \leftarrow Ed.pop\_front()$ 
7       $map\_erasure\_ptr[i - k] \leftarrow D[coding\_data\_map[i - k]]$  init by  $C[i - k]$ 
8  end
9  /* Step 2: Substitute available data into mapping erasures */
10 foreach non-null data chunk  $D[i]$  do
11     foreach non-null  $map\_erasure\_ptr[j]$  do
12         Shift-xor the slices of  $D[i]$  into  $map\_erasure\_ptr[j]$ 
13     end
14 end
15 /* Step 3: Decode erased data and substitute it */
16 foreach non-null  $coding\_data\_map[i]$  do
17     Decode the slice of  $D[coding\_data\_map[i]]$  from  $map\_erasure\_ptr[i]$ 
18     foreach other non-null  $map\_erasure\_ptr[j]$  do
19         Shift-xor the decoded slice into  $map\_erasure\_ptr[j]$ 
20     end
21 end
    
```

为此，提出了 **Cache-Friendly Window** 方案：将较长的数据块划分为更小的窗口（**window**），每次仅处理一个 **window** 的数据。这样可以显著提升数据在 **CPU** 缓存中的命中率，减少反复横跨不同 **chunk** 的内存访问，缓解内存的瓶颈。图3-8举例展示了该方案。同时，每个 **window** 独立初始化，并采用“首次访问时初始化”（**Init On First Access**）的策略。实际应用如 **Ceph** 等分布式存储系统，分配的 **chunk** 内存空间通常不会预先初始化，若一开始就对全部 **chunk** 进行初始化，内存访问的开销很大且有所浪费。因此，结合 **window** 划分方案，只有在 **window** 被实际访问时才进行初始化，边操作边 **init**，很大程度上提升了初始化效率和缓存利用率。

D1: 1 0 1 1 0 0 0 1 1 0 1 1 0 0 0 1

D2: 1 0 1 1 0 0 0 1 1 0 1 1 0 0 0 1

D3: 1 0 1 1 0 0 0 1 1 0 1 1 0 0 0 1

D4: 1 0 1 1 0 0 0 1 1 0 1 1 0 0 0 1

↓ 异或

C1: 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0

在解码过程中，还需考虑 window 与解码流程的特殊关系。Two-tone 码的解码分为“代入”和“消除”两个步骤，不同块之间存在错位。若仅对当前 window 进行“代入”，可能导致解码位置的 window 未被完全覆盖，影响解码正确性。为此，提出 off-by-one 代入策略：在“代入”时，窗口处理范围比“消除”领先一个 window 大小，即“代入”窗口始终覆盖“消除”窗口的全部数据。只需保证 window size 大于数据块个数  $k$ ，即可实现完整覆盖。这一策略易于实现，既保证了缓存局部性，又确保了解码流程的正确性和高效性。图3-8也展示了这种处理方案。

### 3.2.3 内存友好的相邻合并访问：Neighbor-Merging Access

在 Two-tone Shift-XOR 码的实际编码过程中，数据块与校验块的一般访问模式可分为横向访问和纵向访问两种，如图3-9所示。横向访问模式是指每次遍历一个数据块，将该块内的所有数据码字依次异或到所有校验块的对应位置，然后再处理下一个数据块。这种方式对单个数据块的利用非常充分，能够集中地访问和处理数据块，但由于每个数据码字都要异或到所有校验块，导致对校验块的内存需要反复多次访问，且访问不集中，容易造成缓存失效。

纵向访问模式则是针对每个校验块分别进行处理。每次遍历一个校验块，根据该校验块的每个码字，去访问所有数据块上的对应码字并进行异或计算。这样对于同一个校验块来说，只需访问一次即可完成全部计算，提升了单个校验块的访问效率。但由于不同校验块码字的移位偏移量不同，无法复用之前数据块码字异或的结果，导致对数据块的访问变得重复且不集中，影响整体效率。

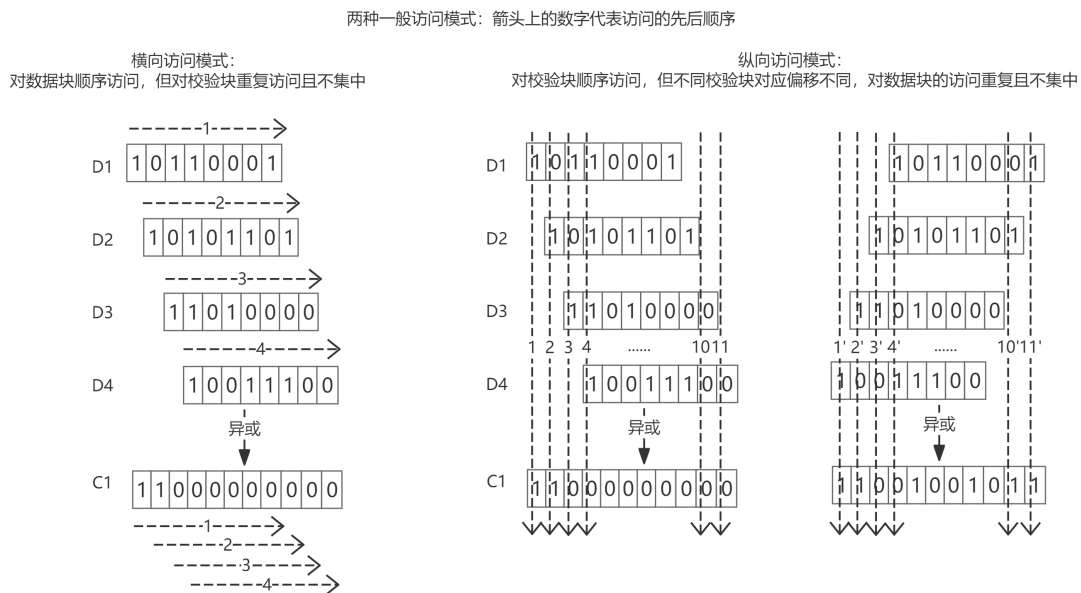


图 3-9 Two-tone 编码横向与纵向访问模式

针对上述两种模式各自的内存访问瓶颈，提出了“相邻合并访问”（Neighbor-Merging Access）策略。该策略结合横向和纵向的优点，将相邻的两个数据块 chunk 的码字在寄存器中合并处理，利用 SIMD 指令或寄存器批量操作，总体上横向访问数据块，计算和访存时则先在寄存器内完成纵向合并运算，再一次性写入校验块内存。这样既能充分利用数据块的访问局部性，又能减少对校验块的重复访问，理论上可减少一半对校验块内存的重复访问，显著提升整体编码效率。



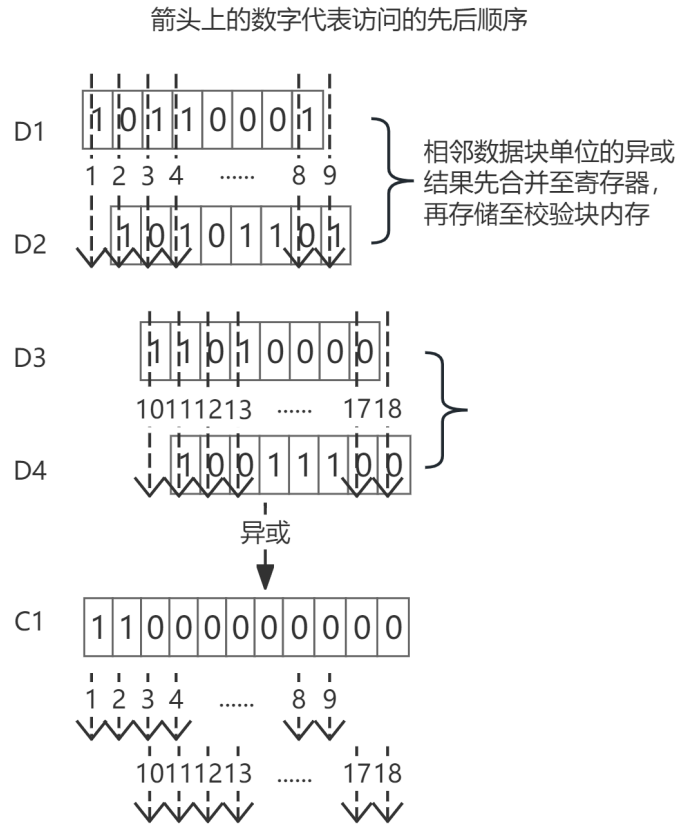


图 3-10 Neighbor-Merging Access 模式

综上，Neighbor-Merging Access 模式通过对横向访问模式与纵向访问模式的有机结合，实现了对数据块和校验块的同时高效利用，减少了对内存的重复和不集中的访问，提升了 Two-tone Shift-XOR 纠删码在实际工程中的编码性能。

### 3.2.4 特化与泛化结合：Special Case Expansibility

在实际分布式存储系统中，多台设备同时发生故障的概率远低于单台设备发生故障的概率，因此只丢失一个块的场景非常得常见。由于这种只丢失一个块的情况丢失模式较少且逻辑简单直观，针对其进行特殊化优化的开发成本很低。

具体而言，当仅有一块数据或者一块校验块发生丢失时，无需采用 Two-tone 移位异或纠删码常规的解码流程，即从两边最外侧错位处向内解码的流程，而是只需要将所有未丢失的相关数据块和校验块整体异或代入即可恢复丢失块，相当于仅执行一个“代入”的步骤，而且只需构建并操作一个 Erasure 结构。图3-11举例展示了这种特殊情况，图例中分别丢失了一个数据块  $D_2$  和一个校验块  $C_3$ 。

这种特化流程不仅极大简化解码逻辑，还具备显著的加速潜力。由于只需访问一次所有 chunk，可以大胆采用 AVX2 等 SIMD 流指令，并利用 stream 命令绕过缓存，进一步提升解码效率。综上，针对单块丢失场景进行特化优化，能够在实际应用中获得极高的性能收益，且与泛化解码流程结合，实现全场景下的高效编解码方案。

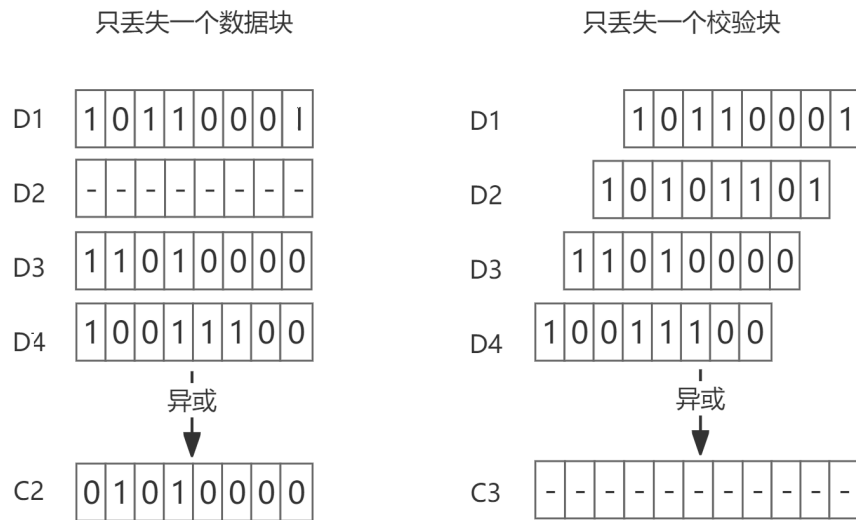


图 3-11 Two-tone 解码只丢失一块的特化情况举例

## 第 4 章 预期目标

### 4.1 系统集成与应用目标

当前大多数纠删码算法仅在自建测试环境或理论分析中验证性能，缺乏在工业级分布式存储系统中的实际集成和应用。为此，本课题将 Two-tone 移位异或纠删码以插件形式集成至 Ceph 分布式存储系统，推动新型高效纠删码技术在真实生产环境中的落地应用。系统集成目标分为如下两个层次：

#### 4.1.1 接口集成与功能验证

首先，通过 Ceph 官方自带的纠删码接口，对 Two-tone 算法进行编码和解码的正确性及性能测试。该阶段重点在于利用 erasure-parity mapping 机制，确保算法能够应对所有可能的数据和校验块丢失情况，满足 Ceph 存储系统的基本应用要求。通过标准化接口测试，验证 Two-tone 算法在多样化擦除模式下的功能完整性和高效性。

#### 4.1.2 集群集成与应用验证

进一步，利用 Ceph 官方自带的集群仿真脚本，建立分布式测试集群，采用自定义 Two-tone 纠删码创建存储池，并在存储池中完成文件的读写操作。该阶段不仅要求算法在单机环境下表现优异，更需在分布式系统中实现稳定可靠的读写流程，标志着 Two-tone 纠删码实现了真正意义上的系统级集成。通过实际应用场景的验证，确保算法具备工业级部署能力和扩展性。

### 4.2 性能优化与方法论总结目标

本课题预期通过提出并实现以下四项系统级优化策略，显著提升 Two-tone 编码和解码的吞吐量，降低内存访问瓶颈，并在实际应用中验证其有效性：

- **统一解码抽象 (Erasure-Parity Mapping)**：实现所有丢失情况的统一处理流程，提升解码和重编码的泛化能力与效率。
- **缓存友好窗口划分 (Cache-Friendly Window)**：通过窗口化处理和首次访问初始化，优化数据块的缓存局部性，减少内存访问开销。
- **内存友好相邻合并访问 (Neighbor-Merging Access)**：结合横向和纵向访问

模式，合并数据块处理，减少对校验块的重复访问，提升整体编码效率。

- **特化优化 (Special Case Expansibility)**: 针对单块丢失等常见场景，采用特化流程和硬件指令集加速，实现极端场景下的高效解码。

上述优化策略的实现和验证是本课题的重要目标。通过与 Ceph 现有 SOTA 纠删码插件（如 ISA-L、Cerasure、ZigZag）进行性能对比，全面评估各项优化措施的实际效果。最终，课题将系统性总结基于移位异或的纠删码系统优化方法论，填补这方面的空白，为后续相关算法的工程实现和产业应用提供参考，推动高效纠删码技术在分布式存储系统中的广泛应用。

## 第 5 章 已完成的研究工作及进度安排

### 5.1 已完成的研究工作

- 梳理并总结国内外纠删码相关理论与系统优化方法，完成文献调研。
- 分析 Two-tone 移位异或纠删码的算法原理，明确优化方向与技术难点。
- 初步设计 Two-tone 编码与解码的统一抽象结构，完成部分解码泛化功能的代码。
- 搭建 Ceph 分布式存储系统的测试环境，集成最简单的 Two-tone 算法实现作为插件，成功通过 Ceph 的纠删码接口测试。

### 5.2 进度安排

1. **2025.09-2025.10**: 继续深入调研相关文献，完善算法理论分析，尝试并明确优化方案。
2. **2025.10-2026.03**: 完整集成 Two-tone 纠删码插件至 Ceph 中，完成 Two-tone 编码与解码算法的高效实现，应用所有优化方案，做消融实验证明各方案有效性，做对比实验证明相较于其他纠删码的高效性。
3. **2026.03-2026.05**: 总结各优化方法论，总结实验成果，撰写论文。

## 第 6 章 已具备的研究条件和所需条件及经费

### 6.1 实验室条件和经费保障

实验室有可用于实验的单台服务器，具备 1TB 容量的 NVMe 磁盘，足够将 Ceph 测试集群单独运行，并进行 GB 级别的大文件读写测试。

### 6.2 所需条件及经费

Ceph 提供了在单台机器上模拟多节点并建立测试集群的功能，与常规多机分布式集群在功能上没有差别，方便做测试研究，也可直接测试纠删码吞吐，因此无需额外的特殊条件与经费。

## 第 7 章 预计困难及解决方案

### 7.1 预计困难与技术难点

本课题的主要困难与技术难点包括：

- Ceph 作为一个发展许久的社区开源项目，内容庞大且复杂，其中的纠删码接口位于顶层应用与底层 IO 之间，难以解析并确定所有相关函数接口及参数的全部含义，可能有实现层面的遗漏。
- Ceph 大型真实分布式集群的环境难以搭建与调试，容易陷入大规模集群维护的困境中，且在物理机完整地源码编译 Ceph 需要繁琐地环境配置与等待，费时费力，不利于课题核心优化方案的研究。
- Ceph 的纠删码接口只会给数据块和校验块申请大小相同的内存空间，然而 Shift-XOR 纠删码的校验块的长度会比数据块大常数级别的大小，在接口内申请额外内存又会导致 Ceph 的内存管理报错。
- Two-tone 算法理论完备，但实现时数据块与校验块的访问模式差异大，泛化解码与重编码的流程依然复杂，难以形成统一的高局部性内存访问策略，需要突破内存瓶颈。

### 7.2 解决方案

针对上述难点，拟采取如下解决措施：

- 借鉴 Ceph 中已经集成好的其他纠删码的实现组织形式，先模仿出一个最简的可运行的自定义纠删码，再逐步丰富；解析 Ceph 提供的纠删码接口测试脚本，确定有哪些重要功能需要被测试，从而指导接口功能的实现。
- Ceph 的特定版本如 v18.2.7 提供了容器环境的配置方案，可以直接在容器中编译 Ceph，无需手动配置环境，并且可保留容器环境用于进一步测试。再结合 Ceph 提供的 vstart 脚本即可在单机的容器内启动多节点虚拟集群，功能同样完善，方便易用，无需再过度担心环境问题。
- 将校验块更长的一小部分数据存储到内存数据库 Redis 中，看作 Ceph 集群管理的元数据之一，由于多出的数据只是常数级别大小，且存储于内存中，对这部分数据的访问几乎不影响纠删码本身的编码和解码性能。
- 关于 Two-tone 系统级别优化难点的解决方案，第 2 章的研究内容与研究方

案中已经做出了充分地阐述，即设计抽象结构、划分数据增加局部性和合并访问内存等方案。



## 参考文献

- [1] FORD D, LABELLE F, POPOVICI F I, et al. Availability in Globally Distributed Storage Systems [C] //9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10), Vancouver, BC : USENIX Association, 2010.
- [2] WEATHERSPOON H, KUBIATOWICZ J D. Erasure Coding Vs. Replication: A Quantitative Comparison [C] //DRUSCHEL P, KAASHOEK F, ROWSTRON A. Peer-to-Peer Systems, Berlin, Heidelberg : Springer Berlin Heidelberg, 2002 : 328-337.
- [3] GHEMAWAT S, GOBIOFF H, LEUNG S-T. The Google File System [C] //Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles, Bolton Landing NY USA : ACM, 2003 : 29-43.
- [4] CALDER B, WANG J, OGUS A, et al. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency [C] //Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles, Cascais Portugal : ACM, 2011 : 143-157.
- [5] HUANG C, SIMITCI H, XU Y, et al. Erasure Coding in Windows Azure Storage [C] //2012 USENIX Annual Technical Conference (USENIX ATC 12), Boston, MA : USENIX Association, 2012 : 15-26.
- [6] SHVACHKO K, KUANG H, RADIA S, et al. The Hadoop Distributed File System [C] //2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), Incline Village, NV, USA : IEEE, 2010 : 1-10.
- [7] CHINIAH A, MUNGUR A. Dynamic Erasure Coding Policy Allocation (DECPA) in Hadoop 3.0 [C] //2019 6th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/ 2019 5th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom), Paris, France : IEEE, 2019 : 29-33.
- [8] WEIL S A, BRANDT S A, MILLER E L, et al. Ceph: A Scalable, High-Performance Distributed File System [C] //7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06), Seattle, WA : USENIX Association, 2006.
- [9] WEIL S A. Ceph: Reliable, Scalable, and High-Performance Distributed Storage [D] . Santa Cruz : University of California, 2007.

- [10] REED I S, SOLOMON G. Polynomial Codes Over Certain Finite Fields [J] . Journal of the Society for Industrial and Applied Mathematics, 1960, 8 (2) : 300-304.
- [11] LUBY M, ZUCKERMANK D. An XOR-Based Erasure-Resilient Coding Scheme [J] . Tech Report, Tech. Rep., 1995.
- [12] PLANK J S, SIMMERMAN S, SCHUMAN C D. Jerasure: A Library in C/C++ Facilitating Erasure Coding for Storage Applications - Version 1.2 : CS-08-627 [R] . Knoxville, TN 37996, USA : University of Tennessee, 2008.
- [13] ARSLAN S S, LE H, LANDMAN J, et al. OpenMP and POSIX Threads Implementation of Jerasure 2.0 [C] //2017 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom), Istanbul, Turkey : IEEE, 2017 : 1-5.
- [14] LIU C, WANG Q, CHU X, et al. G-CRS: GPU Accelerated Cauchy Reed-Solomon Coding [J] . IEEE Transactions on Parallel and Distributed Systems, 2018, 29 (7) : 1484-1498.
- [15] ZHOU T, TIAN C. Fast Erasure Coding for Data Storage: A Comprehensive Study of the Acceleration Techniques [C] //17th USENIX Conference on File and Storage Technologies (FAST 19), Boston, MA : USENIX Association, 2019 : 317-329.
- [16] ZHOU T, TIAN C. Fast Erasure Coding for Data Storage: A Comprehensive Study of the Acceleration Techniques [J] . ACM Transactions on Storage, 2020, 16 (1) : 1-24.
- [17] UEZATO Y. Accelerating XOR-based Erasure Coding Using Program Optimization Techniques [C] //Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis Missouri : ACM, 2021 : 1-14.
- [18] NIU T, LYU M, WANG W, et al. Cerasure: Fast Acceleration Strategies For XOR-Based Erasure Codes [C] //2023 IEEE 41st International Conference on Computer Design (ICCD), Washington, DC, USA : IEEE, 2023 : 535-542.
- [19] WANG W, LYU M, NIU T, et al. Fast Acceleration Strategies for XOR-Based Erasure Codes [J] . IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2025, 44 (1) : 331-344.

- [20] GAJALWAR Y, KHILARI S, KULKARNI H, et al. Exploring Data Protection with ISA-L: A Study on Erasure Coding [C] //2024 Asian Conference on Intelligent Technologies (ACOIT), KOLAR, India : IEEE, 2024 : 1-6.
- [21] SUNG C W, GONG X. A ZigZag-Decodable Code with the MDS Property for Distributed Storage Systems [C] //2013 IEEE International Symposium on Information Theory, Istanbul, Turkey : IEEE, 2013 : 341-345.
- [22] DAI M, SUNG C W, WANG H, et al. A New Zigzag-Decodable Code with Efficient Repair in Wireless Distributed Storage [J] . IEEE Transactions on Mobile Computing, 2017, 16 (5) : 1218-1230.
- [23] GONG X, SUNG C W. Zigzag Decodable Codes: Linear-time Erasure Codes with Applications to Data Storage [J] . Journal of Computer and System Sciences, 2017, 89 : 190-208.
- [24] FU X, XIAO Z, YANG S. Overhead-Free In-place Recovery Scheme for XOR-based Storage Codes [C] //2014 IEEE 13th International Conference on Trust, Security and Privacy in Computing and Communications, Beijing, China : IEEE, 2014 : 552-557.
- [25] FU X, XIAO Z, YANG S. Overhead-Free in-Place Recovery and Repair Schemes of XOR-Based Regenerating Codes [C] //2015 IEEE International Symposium on Information Theory (ISIT), Hong Kong, China : IEEE, 2015 : 854-858.
- [26] FU X, YANG S, XIAO Z. Decoding and Repair Schemes for Shift-XOR Regenerating Codes [J] . IEEE Transactions on Information Theory, 2020, 66 (12) : 7371-7386.
- [27] CHENG X, FU X, GUO Y, et al. Successively Solvable Shift-Add Systems — a Graphical Characterization [C] //2021 IEEE International Symposium on Information Theory (ISIT), Melbourne, Australia : IEEE, 2021 : 946-951.
- [28] FU X, WU C, GUO Y, et al. Two-Tone Shift-XOR Storage Codes [C] //2021 IEEE International Symposium on Information Theory (ISIT), Melbourne, Australia : IEEE, 2021 : 2996-3001.
- [29] DAI M, WANG X, WANG H, et al. Bandwidth Overhead-Free Data Reconstruction Scheme for Distributed Storage Code With Low Decoding Complexity [J] . IEEE Access, 2017, 5 : 6824-6832.
- [30] GONG X, HU P, SHUM K W, et al. A Zigzag-Decodable Ramp Secret Sharing Scheme [J] . IEEE Transactions on Information Forensics and Security, 2018, 13 (8) : 1906-1916.

- [31] KABIRIRAD S, SHEIDANI S, ESLAMI Z. An Efficient Ramp Secret Sharing Scheme Based on Zigzag-Decodable Codes [J] . Computer and Knowledge Engineering, 2023 (Online First) .
- [32] CHEN J, SHEN Y, WAN SUNG C. A New Shift-Add Secret Sharing Scheme for Partial Data Protection With Parallel Zigzag Decoding [J] . IEEE Transactions on Information Forensics and Security, 2024, 19 : 10221-10232.
- [33] DAI M, MAO B, GONG X, et al. Zigzag-Division Multiple Access for Wireless Networks With Long and Heterogeneous Delays [J] . IEEE Transactions on Aerospace and Electronic Systems, 2019, 55 (6) : 2822-2835.
- [34] WEI X, XIE X, CHEN R, et al. Characterizing and Optimizing Remote Persistent Memory with RDMA and NVM [C] // 2021 USENIX Annual Technical Conference (USENIX ATC 21), Online : USENIX Association, 2021 : 523-536.
- [35] JAMES S. PLANK L X. Optimizing Cauchy Reed-Solomon Codes for Fault-Tolerant Network Storage Applications [C] // Fifth IEEE International Symposium on Network Computing and Applications (NCA'06), Cambridge, MA, USA : IEEE, 2006 : 173-180.
- [36] MAKOVENKO M, CHENG M, TIAN C. Revisiting the Optimization of Cauchy Reed-Solomon Coding Matrix for Fault-Tolerant Data Storage [J] . IEEE Transactions on Computers, 2022, 71 (8) : 1839-1846.
- [37] PLANK J S, SCHUMAN C D, ROBISON B D. Heuristics for Optimizing Matrix-Based Erasure Codes for Fault-Tolerant Storage Systems [C] // IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012), Boston, MA, USA : IEEE, 2012 : 1-12.
- [38] LUO J, SHRESTHA M, XU L, et al. Efficient Encoding Schedules for XOR-Based Erasure Codes [J] . IEEE Transactions on Computers, 2014, 63 (9) : 2259-2272.